Find the Promos: solution doc

Rishab Gautam | EE24BTECH11220 | Electrical Engineering (B.Tech), IIT Hyderabad | Problem 1: Find the Promos (Ads)
Prototype: https://abhey-05.github.io/kuku-promo-finder/ | **Repo:** https://github.com/Abhey-05/kuku-promo-finder | **Raw outputs:** link in the submission form

1. Problem choice and framing

I chose Problem 1 because it has one user (the editor), a real input, and an output anyone can check: a timestamp is right or wrong. The cost is concrete too, since an editor watches every episode, every show, every week. Problem 2 carries the risk of invented law, and Problem 3 needs demand evidence I could not gather in 3 hours. I scoped the tool to a shortlist for an editor, not an autoposter. I did not build video analysis, editing, or any posting flow.

2. Assumptions

- Screenplay timestamps match the final video. No video was available to check this.
- A cut may not land inside a spoken line, nor start on the second half of a sentence.
- A hook on the first frame beats the same hook at 2.9 s, so hook cues decay across the window.
- Camera entries count as action: they outnumber action entries about 5 to 1 and often carry the event.
- A public humiliation line counts as a "shocking line" in this genre.
- A late-story clip needs more backstory, so it loses up to 0.3 of its standalone score.
- A stitched promo must share a speaker across joins and stay within 3 episodes.
- The editor sees the top 10, so precision at the top matters more than the order below it.

3. How it works

1. **Parse** each script into timed entries. All 50 episodes parsed with 0 unread lines and 0 timing violations.
2. **Segment** into 20 to 60 s windows on entry boundaries, plus a capped search that stitches strong openings to strong endings: 25,230 single and 332 stitched candidates.
3. **Score** with cue rules. Every score lists the cues behind it. `total = 0.35 hook + 0.35 cliffhanger + 0.15 standalone + 0.15 duration − 0.08 × joins`
4. **Rerank (optional).** The top 30 go to any model behind one interface (Gemini or any OpenAI-compatible endpoint). A reply with an unknown ID or a non-verbatim quote is rejected, and the run falls back to the rules.
5. **Output.** Overlaps are removed (237 left), the top 10 is capped at 2 per episode, and the run writes the ranked promos, every scored candidate, and a log.

Rules do the finding because they are free, repeatable, and explain themselves. The model's job is judgment rules cannot do, such as whether a question matters. It can never add a quote. The web app and CLI share one core.

4. Tools used, and one place each got it wrong

Claude Code wrote the code and drafted the docs. Its cliffhanger word list had "freeze" but not "freezes", so a strong clip ending on a freeze scored 0 and sat at rank 183. Only reading the labelled misses exposed it. Its first ranking also let clips open mid-sentence, which passed every test, and one reached rank 2.

5. Scale to every show

The rules cost nothing: the full pack (102.7 minutes of screen time) ran in under 0.5 s on a laptop and 0.6 s in a browser. The model step sends a fixed top 30, so its cost does not grow with episodes. The real prompt for this pack is 41,554 characters, about 10,400 tokens. With an assumed 1,500 output tokens, at Gemini 2.5 Flash list prices (/bin/zsh.30 in, .50 out per million): 10,400 × $0.30/M + 1,500 × $2.50/M ≈ **$0.007 per show**, or about $0.70 for 100 shows. This is a calculation, not a measured bill, because no key was available and the LLM layer ran only against mocks. A human stays in the loop: the editor reviews the top 10 and thumbs each one, and those labels feed tuning.

6. How I know it's good

Automated: tests enforce that every hook and cliffhanger is verbatim (10/10), every timestamp is valid (25,562/25,562 candidates), promos do not overlap by more than 50%, and runs are deterministic.

Precision: on 36 labels using the brief's three tests, Precision@5 is 80% and Precision@10 is 60%, against an estimated 17.7% for a random pick. **Read these with care:** the labels came from Claude Code, not a human editor; samples are tiny; and the Precision@5 gain from 40% came from fixes tuned on those same labels. Precision@10 stayed at 60%.

Where it picks badly (13 entries in docs/EVAL.md):

- A routine question scores as a cliffhanger, because any ? earns the same credit. This clip is still rank 4.
- Below the top 10 the order is close to noise: post rates ran 0%, 14% and 33% from upper to lower band, the wrong way round.
- The standalone score is the weakest. It rewards a spoken name that means nothing to a newcomer, and passes clips with only supporting characters.
- It missed a clip ending on physical danger (cliffhanger score 0) and has no idea of the premise.

Zeroing the hook weight replaces all of the top 10 and zeroing the cliffhanger replaces 8, so those two carry the ranking.

Once a real team uses it: editor acceptance rate of the top 10, minutes per show against the manual baseline, and an A/B of tool picks against editor picks on install rate per promo.

7. Limits and what's next

- It reads scripts, not video. A clip can read well and play badly. Next: check against shot-level video data.
- The labels need a human editor. Next: 50 editor labels, then retune.
- The model step is untested on a real model. Next: run it with a stronger model and measure it against the same labels.

Decisions in this document are mine; AI assisted with the code and drafting.